

microgpt_dpo.py — An Annotated Guide

Direct Preference Optimization in Pure Python

@nealrichter + Amazon Kiro

June 2026

Abstract

This document walks through `microgpt_dpo.py`, which implements Direct Preference Optimization (DPO) on top of a LoRA-finetuned `microgpt` model. DPO aligns a language model to human preferences without training a separate reward model, using a closed-form loss derived from the RLHF objective. The implementation demonstrates: dual model inference (policy + frozen reference), preference pair training, and the complete DPO loss computation—all in pure Python with zero dependencies.

Source: https://github.com/nealrichter/research/blob/main/microgpt/microgpt_dpo.py

Contents

1	Overview	2
2	The DPO Loss	2
2.1	Derivation	2
3	Dual Model Architecture	3
3.1	Policy Model (with LoRA, differentiable)	3
3.2	Reference Model (plain floats, no grad)	3
4	Log-Probability Computation	3
4.1	Policy (differentiable)	3
4.2	Reference (plain float)	4
5	Preference Data	4
6	DPO Training Loop	4
6.1	Gradient Update	5
7	LoRA Merge and Export	5
8	Usage	5
9	Why DPO Over RLHF?	6

1 Overview

`microgpt_dpo.py` implements the third stage of the post-training pipeline:

1. **Load:** Restore SFT model from `model_sft.json` as both reference policy π_{ref} (frozen) and policy initialization π_{θ} .
2. **Adapt:** Attach LoRA adapters to π_{θ} (Q and V projections).
3. **Train:** Optimize the DPO loss on (prompt, chosen, rejected) triples.
4. **Merge:** Fold adapters into base weights and save `model_dpo.json`.

Parameter	Value
β (KL penalty)	0.3
Reference model	Frozen SFT checkpoint
LoRA rank	4
LoRA alpha	8
Learning rate	0.005
Epochs	30
Preference pairs	140

Table 1: DPO training configuration

2 The DPO Loss

Traditional RLHF [4] requires three stages: SFT, reward model training, and PPO optimization. DPO [1] collapses this into a single loss that directly optimizes the policy on preference data.

2.1 Derivation

Starting from the RLHF objective with KL constraint:

$$\max_{\pi_{\theta}} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(\cdot|x)} [r(x, y)] - \beta \text{KL} [\pi_{\theta}(\cdot|x) \parallel \pi_{\text{ref}}(\cdot|x)]$$

Rafailov et al. show the optimal policy has a closed form:

$$\pi^*(y|x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y|x) \exp\left(\frac{r(x, y)}{\beta}\right)$$

Rearranging gives the implicit reward:

$$r(x, y) = \beta \log \frac{\pi_{\theta}(y|x)}{\pi_{\text{ref}}(y|x)} + \beta \log Z(x)$$

Substituting into the Bradley-Terry preference model and canceling $Z(x)$:

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma \left(\beta \left(\log \frac{\pi_{\theta}(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \log \frac{\pi_{\theta}(y_l|x)}{\pi_{\text{ref}}(y_l|x)} \right) \right) \right]$$

where y_w is the preferred response, y_l is the rejected response, and σ is the sigmoid function.

3 Dual Model Architecture

DPO requires two forward passes per response: one through the trainable policy π_θ and one through the frozen reference π_{ref} .

3.1 Policy Model (with LoRA, differentiable)

```
80 state_dict = {k: [[Value(w) for w in row] for row in mat]
81                  for k, mat in model['weights'].items()}
82
83 base_params = [p for mat in state_dict.values()
84                 for row in mat for p in row]
```

The policy uses Value nodes for automatic differentiation, with LoRA adapters on Q and V:

```
129 def gpt_policy(token_id, pos_id, keys, values):
130     ...
131     q = linear_lora(x, state_dict[f'layer{li}.attn_wq'],
132                     lora_dict[f'layer{li}.attn_wq.down'],
133                     lora_dict[f'layer{li}.attn_wq.up'])
134     k = linear(x, state_dict[f'layer{li}.attn_wk'])
135     v = linear_lora(x, state_dict[f'layer{li}.attn_wv'],
136                     lora_dict[f'layer{li}.attn_wv.down'],
137                     lora_dict[f'layer{li}.attn_wv.up'])
```

3.2 Reference Model (plain floats, no grad)

For efficiency, the reference model operates on raw floats—no computation graph needed since we never backpropagate through it:

```
91 ref_weights = {k: [[w for w in row] for row in mat]
92                   for k, mat in model['weights'].items()}

160 def linear_ref(x, w):
161     return [sum(wo[j] * x[j] for j in range(len(x))) for wo in w]
162
163 def gpt_ref(token_id, pos_id, keys, values):
164     ... # identical architecture, plain float arithmetic
```

This is the key insight: the reference model is **frozen**, so we never need gradients through it. Using plain floats makes it $\sim 3\text{--}5\times$ faster than the policy forward pass.

4 Log-Probability Computation

Both models compute $\log P(y|x)$ as the sum of per-token log probabilities over the response:

$$\log \pi(y|x) = \sum_{t=1}^{|y|} \log P(y_t \mid x, y_{<t})$$

4.1 Policy (differentiable)

```
205 def log_prob_policy(prompt_tok, resp_tok):
206     tokens = prompt_tok + resp_tok
207     n = min(block_size, len(tokens) - 1)
208     keys, values = [[] for _ in range(n_layer)],
```

```

209         [[] for _ in range(n_layer)]
210     log_p = Value(0.0)
211     resp_start = len(prompt_tok)
212     for pos_id in range(n):
213         logits = gpt_policy(tokens[pos_id], pos_id, keys, values)
214         if pos_id >= resp_start - 1 and pos_id < len(tokens) - 1:
215             probs = softmax(logits)
216             log_p = log_p + probs[tokens[pos_id + 1]].log()
217     return log_p

```

4.2 Reference (plain float)

```

219 def log_prob_ref(prompt_tok, resp_tok):
220     tokens = prompt_tok + resp_tok
221     n = min(block_size, len(tokens) - 1)
222     keys, values = [[] for _ in range(n_layer)],
223                    [[] for _ in range(n_layer)]
224     log_p = 0.0
225     resp_start = len(prompt_tok)
226     for pos_id in range(n):
227         logits = gpt_ref(tokens[pos_id], pos_id, keys, values)
228         if pos_id >= resp_start - 1 and pos_id < len(tokens) - 1:
229             probs = softmax_ref(logits)
230             log_p += math.log(probs[tokens[pos_id + 1]] + 1e-10)
231     return log_p

```

5 Preference Data

DPO requires (prompt, chosen, rejected) triples. The data file uses pipe-delimited format:

```

fa|Alice|Bob
mb|Benjamin|Alice
fc|Charlotte|Daniel

```

```

235 dpo_data = []
236 if os.path.exists('input_dpo.txt'):
237     for line in open('input_dpo.txt'):
238         parts = line.strip().split('|')
239         if len(parts) == 3:
240             dpo_data.append(tuple(parts))

```

The chosen response satisfies the instruction constraints (correct gender + starting letter); the rejected response violates them.

6 DPO Training Loop

The core training computes the DPO loss for each preference pair:

```

319     # Policy log probs (differentiable)
320     log_pi_chosen = log_prob_policy(prompt_tok, chosen_tok)
321     log_pi_rejected = log_prob_policy(prompt_tok, rejected_tok
322                                     )
323     # Reference log probs (frozen, float)

```

```

324     log_ref_chosen = log_prob_ref(prompt_tok, chosen_tok)
325     log_ref_rejected = log_prob_ref(prompt_tok, rejected_tok)
326
327     # DPO loss
328     reward_chosen = log_pi_chosen + Value(-log_ref_chosen)
329     reward_rejected = log_pi_rejected + Value(-
        log_ref_rejected)
330     logit = (reward_chosen + (reward_rejected * -1)) * beta
331     loss = (sigmoid(logit).log()) * -1

```

This implements:

$$\mathcal{L} = -\log \sigma \left(\beta \cdot \underbrace{(\log \pi_{\theta}(y_w|x) - \log \pi_{\text{ref}}(y_w|x))}_{\text{chosen reward}} - \beta \cdot \underbrace{(\log \pi_{\theta}(y_l|x) - \log \pi_{\text{ref}}(y_l|x))}_{\text{rejected reward}} \right)$$

The loss decreases when the policy assigns relatively higher probability to the chosen response (compared to the reference) than to the rejected response.

6.1 Gradient Update

Only LoRA parameters are updated; base model gradients are zeroed:

```

335     lr_t = lr * (1 - step / total_steps)
336     for i, p in enumerate(lora_params):
337         m[i] = beta1 * m[i] + (1 - beta1) * p.grad
338         v_buf[i] = beta2 * v_buf[i] + (1 - beta2) * p.grad**2
339         p.data -= lr_t * (m[i] / (1 - beta1**(step+1))) \
340             / ((v_buf[i] / (1 - beta2**(step+1)))**0.5 + eps)
341         p.grad = 0
342     for p in base_params:
343         p.grad = 0

```

7 LoRA Merge and Export

Identical to SFT—adapters are folded into base weights:

$$W_{\text{merged}} = W_0 + \frac{\alpha}{r} \cdot W_{\text{up}} \cdot W_{\text{down}}$$

```

351     for i in range(n_layer):
352         for key in (f'layer{i}.attn_wq', f'layer{i}.attn_wv'):
353             w_base = state_dict[key]
354             w_down = lora_dict[f'{key}.down']
355             w_up = lora_dict[f'{key}.up']
356             for r in range(len(w_base)):
357                 for c in range(len(w_base[0])):
358                     w_base[r][c].data += lora_scale * sum(
359                         w_up[r][k].data * w_down[k][c].data
360                         for k in range(lora_rank))

```

The merged model is saved as `model_dpo.json`, compatible with `microgpt.py -i model_dpo.json`.

8 Usage

```

1 # Step 1: Pretrain base model
2 python3 microgpt.py
3
4 # Step 2: LoRA SFT
5 python3 microgpt_sft.py
6
7 # Step 3: DPO alignment (loads model_sft.json, saves model_dpo.
8   json)
9 python3 microgpt_dpo.py
10
11 # Inference from DPO model
12 python3 microgpt_dpo.py -i
python3 microgpt.py -i model_dpo.json

```

9 Why DPO Over RLHF?

	RLHF (PPO)	DPO
Stages	3 (SFT $\rightarrow$ RM $\rightarrow$ PPO)	1
Reward model	Required	Implicit
Training stability	Sensitive to hyperparams	Stable
Memory	4 models in memory	2 models
Theoretical guarantees	Same	Same

Table 2: RLHF vs. DPO comparison (from Rafailov et al. [1])

As noted by Tie et al. [2]: “DPO collapses the 3-stage RLHF pipeline into a single-stage optimization directly on preference data, bypassing explicit reward modeling entirely.”

References

- [1] Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C.D., and Finn, C. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. Advances in Neural Information Processing Systems 36, 2023. <https://arxiv.org/abs/2305.18290>
- [2] Tie, G., Zhao, Z., Song, D., et al. *A Survey on Post-training of Large Language Models*. arXiv:2503.06072v3, March 2025. <https://arxiv.org/abs/2503.06072>
- [3] Hu, J.E., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., and Chen, W. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv:2106.09685, 2021. <https://arxiv.org/abs/2106.09685>
- [4] Ouyang, L., Wu, J., Jiang, X., et al. *Training Language Models to Follow Instructions with Human Feedback*. Advances in Neural Information Processing Systems 35, 2022. <https://arxiv.org/abs/2203.02155>
- [5] Kingma, D.P. and Ba, J. *Adam: A Method for Stochastic Optimization*. Proceedings of ICLR, 2015. <https://arxiv.org/abs/1412.6980>